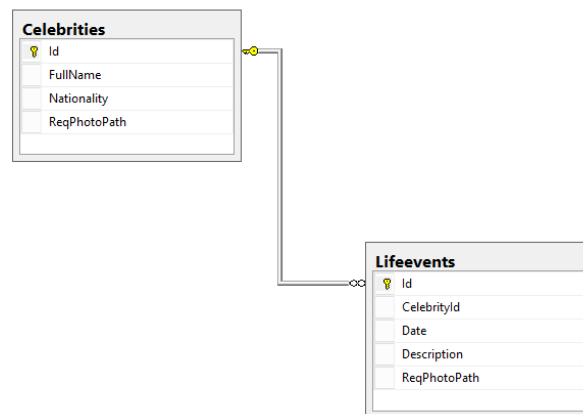


ASPA – приложение ASP.NET CORE

♣ – задания, требующие самостоятельного изучения (нет в лекциях)

Задание 1.

1. Исследуйте логическую схему базы данных «Life Events of Celebrities» (LES) и поясняющие назначение полей классы, соответствующие (ORM) таблицам LES.



```
public class Celebrity // Знаменитость
{
    public Celebrity() { this.FullName = string.Empty; this.Nationality = string.Empty; }
    public int Id { get; set; } // Id Знаменитости
    public string FullName { get; set; } // полное имя Знаменитости
    public string Nationality { get; set; } // гражданство Знаменитости ( 2 символа ISO )
    public string? ReqPhotoPath { get; set; } // request path Фотографии
    public virtual bool Update(Celebrity celebrity) // --вспомогательный метод ...
}

public class Lifevent // Событие в жизни знаменитости
{
    public Lifevent() { this.Description = string.Empty; }
    public int Id { get; set; } // Id События
    public int CelebrityId { get; set; } // Id Знаменитости
    public DateTime Date { get; set; } // дата События
    public string Description { get; set; } // описание События
    public string? ReqPhotoPath { get; set; } // request path Фотографии
    public virtual bool Update(Lifevent lifevent) // -- вспомогательный метод
}
```

Задание 2.

2. Разработайте библиотеку **DAL_Celebrity** содержащую следующие интерфейсы (прилагается файл **DAL_Celebrity.cs**)

```
public interface IRepository<T1, T2> : IMix<T1, T2>, ICelebrity<T1>, ILifeevent<T2> { }

public interface IMix<T1, T2>
{
    List<T2> GetLifeeventsByCelebrityId(int celebrityId); // получить все События по Id Знаменитости
    T1? GetCelebrityByLifeeventId(int lifeeventId); // получить Знаменитость по Id События
}

public interface ICelebrity<T> : IDisposableable
{
    List<T> GetAllCelebrities(); // получить все Знаменитости
    T? GetCelebrityById(int Id); // получить Знаменитость по Id
    bool DelCelebrity(int id); // удалить Знаменитость по Id
    bool AddCelebrity(T celebrity); // добавить Знаменитость
    bool UpdCelebrity(int id, T celebrity); // изменить Знаменитость по Id
    int GetCelebrityIdByName(string name); // получить первый Id по вхождению подстроки
}

public interface ILifeevent<T> : IDisposableable
{
    List<T> GetAllLifeevents(); // получить все События
    T? GetLifeeventById(int Id); // получить Событие по Id
    bool DelLifeevent(int id); // удалить Событие по Id
    bool AddLifeevent(T lifeevent); // добавить Событие
    bool UpdLifeevent(int id, T lifeevent); // изменить событие по Id
}
```

3. Разработайте библиотеку **DAL_Celebrity_MSSQL** реализующую интерфейс **IRepository** (**DAL_Celebrity**) с применением Entity Framework (NuGet-пакет).

```
public interface IRepository: DAL_Celebrity.IRepository<Celebrity, Lifeevent> { }

public class Celebrity // Знаменитость
{
    public Celebrity() { this.FullName = string.Empty; this.Nationality = string.Empty; }
    public int Id { get; set; } // Id Знаменитости
    public string FullName { get; set; } // полное имя Знаменитости
    public string Nationality { get; set; } // гражданство Знаменитости ( 2 символа ISO )
    public string? ReqPhotoPath { get; set; } // request path Фотографии
    public virtual bool Update(Celebrity celebrity) // --вспомогательный метод ...
}

public class Lifeevent // Событие в жизни знаменитости
{
    public Lifeevent() { this.Description = string.Empty; }
    public int Id { get; set; } // Id События
    public int CelebrityId { get; set; } // Id Знаменитости
    public DateTime? Date { get; set; } // дата События
    public string Description { get; set; } // описание События
    public string? ReqPhotoPath { get; set; } // request path Фотографии
    public virtual bool Update(Lifeevent lifeevent) // -- вспомогательный метод
}
```

```

public class Repository : IRepository
{
    Context context;
    public Repository() { this.context = new Context(); }
    public Repository(string connectionString) { this.context = new Context(connectionString); }
    public static IRepository Create() { return new Repository(); }
    public static IRepository Create(string connectionString) { return new Repository(connectionString); }
    public List<Celebrity> GetAllCelebrities() { return this.context.Celebrities.ToList<Celebrity>(); }
    public Celebrity? GetCelebrityById(int Id) {...}
    public bool AddCelebrity(Celebrity celebrity) {...}
    public bool DelCelebrity(int id) {...}
    public bool UpdCelebrity(int id, Celebrity celebrity) {...}
    public List<Lifeevent> GetAllLifeevents() { return this.context.Lifeevents.ToList<Lifeevent>(); }
    public Lifeevent? GetLifeeventById(int Id) {...}
    public bool AddLifeevent(Lifeevent lifeevent) {...}
    public bool DelLifeevent(int id) {...}
    public bool UpdLifeevent(int id, Lifeevent lifeevent) {...}
    public List<Lifeevent> GetLifeeventsByCelebrityId(int celebrityId) {...}
    public Celebrity? GetCelebrityByLifeeventId(int lifeeventId) {...}
    public int GetCelebrityIdByName(string name) {...}
    public void Dispose() {...}
}

```

```

public class Context : DbContext
{
    public string? ConnectionString {get; private set;} = null;
    public Context(string connstring) : base()
    {
        this.ConnectionString = connstring;
        //this.Database.EnsureDeleted();
        //this.Database.EnsureCreated();
    }
    public Context() : base()
    {
        //this.Database.EnsureDeleted();
        //this.Database.EnsureCreated();
    }
    public DbSet<Celebrity> Celebrities { get; set; }
    public DbSet<Lifeevent> Lifeevents { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        if (this.ConnectionString is null) this.ConnectionString = @"Data source=.;Initial Catalog=celebrity;Integrated Security=True;User=sa;Password=1qaz!@WSXxcde";
        optionsBuilder.UseSqlServer(this.ConnectionString);
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Celebrity>()..ToTable("Celebrities").HasKey(p => p.Id);
        modelBuilder.Entity<Celebrity>().Property(p => p.Id).IsRequired();
        modelBuilder.Entity<Celebrity>().Property(p => p.FullName).IsRequired().HasMaxLength(50);
        modelBuilder.Entity<Celebrity>().Property(p => p.Nationality).IsRequired().HasMaxLength(20);
        modelBuilder.Entity<Celebrity>().Property(p => p.RealPhotoPath).HasMaxLength(200);

        modelBuilder.Entity<Lifeevent>()..ToTable("Lifeevents").HasKey(p => p.Id);
        modelBuilder.Entity<Lifeevent>().Property(p => p.Id).IsRequired();
        modelBuilder.Entity<Lifeevent>().Property(p => p.CelebrityId).IsRequired().HasForeignKey(p => p.CelebrityId);
        modelBuilder.Entity<Lifeevent>().Property(p => p.Date);
        modelBuilder.Entity<Lifeevent>().Property(p => p.Description).HasMaxLength(256);
        modelBuilder.Entity<Lifeevent>().Property(p => p.RealPhotoPath).HasMaxLength(256);
        base.OnModelCreating(modelBuilder);
    }
}

```

4. Добавьте в библиотеку **DAL_Celebrity_MSSQL** класс **Init**, предназначенный для инициализации и тестирования БД (файл **Init.cs** прилагается).

```

public class Init
{
    static string connstring = @"Data source=172.16.193.88; Initial Catalog=LES01;"+
                                @"TrustServerCertificate=True;User Id=smw60;Password=21625";

    public Init() { }
    public Init( string conn) { connstring = conn;}
    public static void Execute(bool delete = true, bool create = true)
    {
        Context context = new Context(connstring);
        if (delete) context.Database.EnsureDeleted(); // если есть БД, то она удаляется
        if (create) context.Database.EnsureCreated(); // если нет БД, то создается

        Func<string, string> puri = (string f) => $"{{{f}}}";

        { //1
            Celebrity c = new Celebrity() { FullName = "Noam Chomsky", Nationality = "US", ReqPhotoPath = puri("Chomsky.jpg") };
            Lيفةvent l1 = new Lيفةvent() { CelebrityId = 1, Date = new DateTime(1928, 12, 7), Description = "Дата рождения", ReqPhotoPath = null };
            Lيفةvent l2 = new Lيفةvent() { CelebrityId = 1, Date = new DateTime(1955, 1, 1),
                                           Description = "Издание книги \\"Логическая структура лингвистической теории\\" ", ReqPhotoPath = null };
            context.Celebrities.Add(c);
            context.Lifeevents.Add(l1);
            context.Lifeevents.Add(l2);
        }

        { //2
            Celebrity c = new Celebrity() { FullName = "Tim Berners-Lee", Nationality = "UK", ReqPhotoPath = puri("Berners-Lee.jpg") };
            Lيفةvent l1 = new Lيفةvent() { CelebrityId = 2, Date = new DateTime(1955, 6, 8), Description = "Дата рождения", ReqPhotoPath = null };
            Lيفةvent l2 = new Lيفةvent() { CelebrityId = 2, Date = new DateTime(1989, 6, 8),
                                           Description = "В CERN предложил \\"Гипертекстовый проект\\" ", ReqPhotoPath = null };
            context.Celebrities.Add(c);
            context.Lifeevents.Add(l1);
            context.Lifeevents.Add(l2);
        }
    } //3

    { //3
        Celebrity c = new Celebrity() { FullName = "Edgar Codd", Nationality = "US", ReqPhotoPath = puri("Codd.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 3, Date = new DateTime(1923, 8, 23), Description = "Дата рождения", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 3, Date = new DateTime(2003, 4, 18), Description = "Дата смерти", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }

    { //4
        Celebrity c = new Celebrity() { FullName = "Donald Knuth", Nationality = "US", ReqPhotoPath = puri("Knuth.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 4, Date = new DateTime(1938, 1, 10), Description = "Дата рождения", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 4, Date = new DateTime(1974, 1, 1), Description = "Премия Тьюринга", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }

    { //5
        Celebrity c = new Celebrity() { FullName = "Linus Torvalds", Nationality = "US", ReqPhotoPath = puri("Linus.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 5, Date = new DateTime(1969, 12, 28 ),
                                       Description = "Дата рождения. Финляндия.", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 5, Date = new DateTime(1991, 9, 17),
                                       Description = "Выложил исходный код OS Linux (версии 0.01)", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }

    { //6
        Celebrity c = new Celebrity() { FullName = "John Neumann", Nationality = "US", ReqPhotoPath = puri("Neumann.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 6, Date = new DateTime(1903, 12, 28),
                                       Description = "Дата рождения. Венгрия", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 6, Date = new DateTime(1957, 2, 8), Description = "Дата смерти", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }

    { //7
        Celebrity c = new Celebrity() { FullName = "Edsger Dijkstra", Nationality = "NL", ReqPhotoPath = puri("Dijkstra.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 7, Date = new DateTime(1930, 12, 28), Description = "Дата рождения", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 7, Date = new DateTime(2002, 8, 6), Description = "Дата смерти", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }

    { //8
        Celebrity c = new Celebrity() { FullName = "Ada Lovelace", Nationality = "UK", ReqPhotoPath = puri("Lovelace.jpg") };
        Lيفةvent l1 = new Lيفةvent() { CelebrityId = 8, Date = new DateTime(1852, 11, 27), Description = "Дата рождения", ReqPhotoPath = null };
        Lيفةvent l2 = new Lيفةvent() { CelebrityId = 8, Date = new DateTime(1815, 12, 10), Description = "Дата смерти", ReqPhotoPath = null };
        context.Celebrities.Add(c);
        context.Lifeevents.Add(l1);
        context.Lifeevents.Add(l2);
    }
}

```

```

{ //9
    Celebrity c = new Celebrity() { FullName = "Charles Babbage", Nationality = "UK", ReqPhotoPath = puri("Babbage.jpg") };
    Lifeevent l1 = new Lifeevent() { CelebrityId = 9, Date = new DateTime(1791, 12, 26), Description = "Дата рождения", ReqPhotoPath = null };
    Lifeevent l2 = new Lifeevent() { CelebrityId = 9, Date = new DateTime(1871, 10, 18), Description = "Дата смерти", ReqPhotoPath = null };
    context.Celebrities.Add(c);
    context.Lifeevents.Add(l1);
    context.Lifeevents.Add(l2);
}
{ //10
    Celebrity c = new Celebrity() { FullName = "Andrew Tanenbaum", Nationality = "NL", ReqPhotoPath = puri("Tanenbaum.jpg") };
    Lifeevent l1 = new Lifeevent() { CelebrityId = 10, Date = new DateTime(1944, 3, 16), Description = "Дата рождения", ReqPhotoPath = null };
    Lifeevent l2 = new Lifeevent() { CelebrityId = 10, Date = new DateTime(1987, 1, 1),
        Description = "Создал OS MINIX - бесплатную Unix-подобную систему", ReqPhotoPath = null };

    context.Celebrities.Add(c);
    context.Lifeevents.Add(l1);
    context.Lifeevents.Add(l2);
}
context.SaveChanges();
}

```

Задание 3

5. Разработайте консольное приложение ***DAL_Celebrity_MSSQL_Test*** для тестирования репозитория (из ***DAL_Celebrity_MSSQL***).
6. Программный код теста (файл ***Programm.cs*** прилагается).

```

using DAL_Celebrity_MSSQL;
internal class Program
{
    private static void Main(string[] args)
    {
        string CS = @"Data source=.; Initial Catalog=LES01;TrustServerCertificate=True;User Id=.;Password=.";

        Init init = new Init(CS);
        Init.Execute(delete:true, create:true);

        Func<Celebrity, string> printC = (c) => $"Id = {c.Id}, FullName = {c.FullName}, Nationality = {c.Nationality}, ReqPhotoPath = {c.ReqPhotoPath}";
        Func<Lifeevent, string> printL = (l) => $"Id = {l.Id}, CelebrityId = {l.CelebrityId}, Date = {l.Date}, Description = {l.Description}, ReqPhotoPath = {l.ReqPhotoPath}";
        Func<string, string> puri = (string f) => $"{{{f}}}";
    }
}

```

```

using (IRepository repo = Repository.Create(CS))
{
    { Console.WriteLine("----- GetAllCelebrities() ----- ");
      repo.GetAllCelebrities().ForEach(celebrity => Console.WriteLine(printC(celebrity)));
    }
    { Console.WriteLine("----- GetAllLifeevents() ----- ");
      repo.GetAllLifeevents().ForEach(life => Console.WriteLine(printL(life)));
    }
    { Console.WriteLine("----- AddCelebrity() ----- ");
      Celebrity c = new Celebrity { FullName = "Albert Einstein", Nationality = "DE", ReqPhotoPath = puri("Einstein.jpg") };
      if (repo.AddCelebrity(c)) Console.WriteLine($"OK: AddCelebrity: {printC(c)}");
      else Console.WriteLine($"ERROR: AddCelebrity: {printC(c)}");
    }
    { Console.WriteLine("----- AddCelebrity() ----- ");
      Celebrity c = new Celebrity { FullName = "Samuel Huntington", Nationality = "US", ReqPhotoPath = puri("Huntington.jpg") };
      if (repo.AddCelebrity(c)) Console.WriteLine($"OK: AddCelebrity: {printC(c)}");
      else Console.WriteLine($"ERROR: AddCelebrity: {printC(c)}");
    }
    { Console.WriteLine("----- DelCelebrity() ----- ");
      int id = 0;
      if ((id = repo.GetCelebrityIdByName("Einstein")) > 0) ...
      else Console.WriteLine($"ERROR: GetCelebrityIdByName");
    }
    { Console.WriteLine("----- UpdCelebrity() ----- ");
      int id = 0;
      if ((id = repo.GetCelebrityIdByName("Huntington")) > 0)
      {
          Celebrity? c = repo.GetCelebrityById(id);
          if (c != null) ...
          else Console.WriteLine($"ERROR: GetCelebrityById: {id}");
      }
      else Console.WriteLine($"ERROR: GetCelebrityIdByName");
    }
    { Console.WriteLine("----- AddLifeevent() ----- ");
      int id = 0;
      if ((id = repo.GetCelebrityIdByName("Huntington")) > 0) ...
      else Console.WriteLine($"ERROR: GetCelebrityIdByName");
    }
}

```



```

{ Console.WriteLine("----- DelLifeevent() ----- ");
  int id = 22;
  if (repo.DelLifeevent(id)) Console.WriteLine($"OK: DelLifeevent: {id}");
  else Console.WriteLine($"ERROR: DelLifeevent: {id}");
}
{ Console.WriteLine("----- UpdLifeevent() ----- ");
  int id = 23;
  Lifeevent? ll;
  if ((ll = repo.GetLifeeventById(id)) != null)
  {
    ll.Description = "Дата смерти";
    if (repo.UpdLifeevent(id, ll)) Console.WriteLine($"OK: UpdLifeevent {id}, {printL(ll)}");
    else Console.WriteLine($"ERROR: UpdLifeevent {id}, {printL(ll)}");
  }
}
{ Console.WriteLine("----- GetLifeeventsByCelebrityId ----- ");
  int id = 0;
  if ((id = repo.GetCelebrityIdByName("Huntington")) > 0)
  {
    Celebrity? c = repo.GetCelebrityById(id);
    if (c != null) repo.GetLifeeventsByCelebrityId(c.Id).ForEach(l => Console.WriteLine($"OK: GetLifeeventsByCelebrityId, {id}, {printL(l)}"));
    else Console.WriteLine($"ERROR: GetLifeeventsByCelebrityId: {id}");
  }
  else Console.WriteLine($"ERROR: GetCelebrityIdByName");
}
{ Console.WriteLine("----- GetCelebrityByLifeeventId ----- ");
  int id = 23;
  Celebrity? c;
  if ((c = repo.GetCelebrityByLifeeventId(id)) != null) Console.WriteLine($"OK: {printC(c)}");
  else Console.WriteLine($"ERROR: GetCelebrityByLifeeventId, {id}");
}
}
Console.WriteLine("----->"); Console.ReadKey();
}

```

7. Результаты теста

```

----- GetAllCelebrities() -----
Id = 1, FullName = Noam Chomsky, Nationality = US, ReqPhotoPath = Chomsky.jpg
Id = 2, FullName = Tim Berners-Lee, Nationality = UK, ReqPhotoPath = Berners-Lee.jpg
Id = 3, FullName = Edgar Codd, Nationality = US, ReqPhotoPath = Codd.jpg
Id = 4, FullName = Donald Knuth, Nationality = US, ReqPhotoPath = Knuth.jpg
Id = 5, FullName = Linus Torvalds, Nationality = US, ReqPhotoPath = Linus.jpg
Id = 6, FullName = John Neumann, Nationality = US, ReqPhotoPath = Neumann.jpg
Id = 7, FullName = Edsger Dijkstra, Nationality = NL, ReqPhotoPath = Dijkstra.jpg
Id = 8, FullName = Ada Lovelace, Nationality = UK, ReqPhotoPath = Lovelace.jpg
Id = 9, FullName = Charles Babbage, Nationality = UK, ReqPhotoPath = Babbage.jpg
Id = 10, FullName = Andrew Tanenbaum, Nationality = NL, ReqPhotoPath = Tanenbaum.jpg

----- GetAllLifeevents() -----
Id = 1, CelebrityId = 1, Date = 07.12.1928 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 2, CelebrityId = 1, Date = 01.01.1955 0:00:00, Description = Издание книги "Логическая структура лингвистической теории", ReqPhotoPath =
Id = 3, CelebrityId = 2, Date = 08.06.1955 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 4, CelebrityId = 2, Date = 08.06.1989 0:00:00, Description = В CERN предложил "Гипертекстовый проект", ReqPhotoPath =
Id = 5, CelebrityId = 3, Date = 23.08.1923 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 6, CelebrityId = 3, Date = 18.04.2003 0:00:00, Description = Дата смерти, ReqPhotoPath =
Id = 7, CelebrityId = 4, Date = 10.01.1938 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 8, CelebrityId = 4, Date = 01.01.1974 0:00:00, Description = Премия Тьюринга, ReqPhotoPath =
Id = 9, CelebrityId = 5, Date = 28.12.1969 0:00:00, Description = Дата рождения. Финляндия., ReqPhotoPath =
Id = 10, CelebrityId = 5, Date = 17.09.1991 0:00:00, Description = Выложил исходный код OS Linux (версии 0.01), ReqPhotoPath =
Id = 11, CelebrityId = 6, Date = 28.12.1903 0:00:00, Description = Дата рождения. Венгрия, ReqPhotoPath =
Id = 12, CelebrityId = 6, Date = 08.02.1957 0:00:00, Description = Дата смерти, ReqPhotoPath =
Id = 13, CelebrityId = 7, Date = 28.12.1930 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 14, CelebrityId = 7, Date = 06.08.2002 0:00:00, Description = Дата смерти, ReqPhotoPath =
Id = 15, CelebrityId = 8, Date = 27.11.1852 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 16, CelebrityId = 8, Date = 10.12.1815 0:00:00, Description = Дата смерти, ReqPhotoPath =
Id = 17, CelebrityId = 9, Date = 26.12.1791 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 18, CelebrityId = 9, Date = 18.10.1871 0:00:00, Description = Дата смерти, ReqPhotoPath =
Id = 19, CelebrityId = 10, Date = 16.03.1944 0:00:00, Description = Дата рождения, ReqPhotoPath =
Id = 20, CelebrityId = 10, Date = 01.01.1987 0:00:00, Description = Создал OS MINIX - бесплатную Unix-подобную систему, ReqPhotoPath =

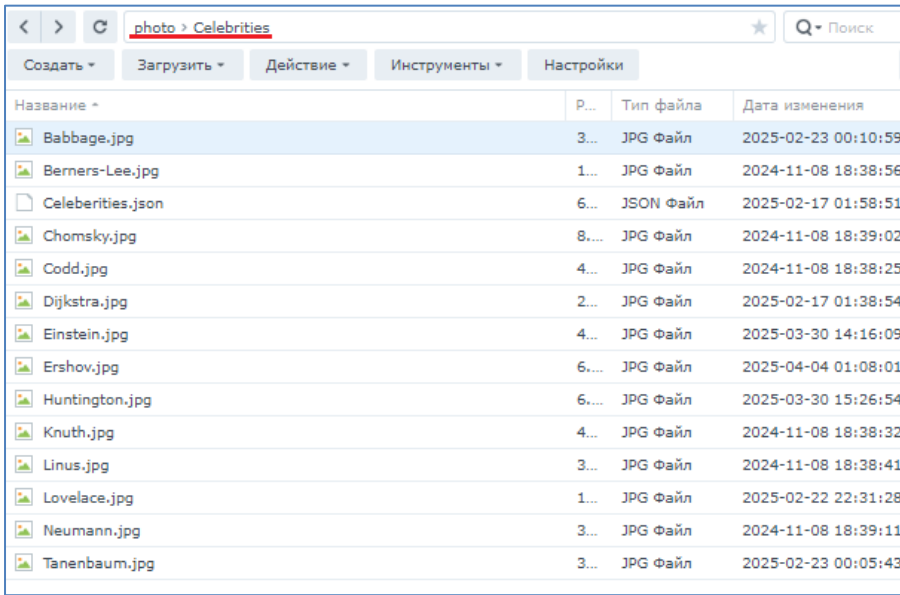
----- AddCelebrity() -----
OK: AddCelebrity: Id = 11, FullName = Albert Einstein, Nationality = DE, ReqPhotoPath = Einstein.jpg
----- AddCelebrity() -----
OK: AddCelebrity: Id = 12, FullName = Samuel Huntington, Nationality = US, ReqPhotoPath = Huntington.jpg
----- DelCelebrity() -----
OK: DelCelebrity: 11
----- UpdCelebrity() -----
OK: UpdCelebrity:12, Id = 12, FullName = Samuel Phillips Huntington, Nationality = US, ReqPhotoPath = Huntington.jpg
OK: GetCelebrityById, Id = 12, FullName = Samuel Phillips Huntington, Nationality = US, ReqPhotoPath = Huntington.jpg
----- AddLifeevent() -----
OK: AddLifeevent, Id = 21, CelebrityId = 12, Date = 18.04.1927 0:00:00, Description = Дата рождения, ReqPhotoPath =
OK: AddLifeevent, Id = 22, CelebrityId = 12, Date = 18.04.1927 0:00:00, Description = Дата рождения, ReqPhotoPath =
OK: AddLifeevent, Id = 23, CelebrityId = 12, Date = 24.12.2008 0:00:00, Description = Дата рождения, ReqPhotoPath =
----- DelLifeevent() -----
OK: DelLifeevent: 22
----- UpdLifeevent() -----
OK: UpdLifeevent 23, Id = 23, CelebrityId = 12, Date = 24.12.2008 0:00:00, Description = Дата смерти, ReqPhotoPath =
----- GetLifeeventsByCelebrityId -----
OK: GetLifeeventsByCelebrityId, 12, Id = 21, CelebrityId = 12, Date = 18.04.1927 0:00:00, Description = Дата рождения, ReqPhotoPath =
OK: GetLifeeventsByCelebrityId, 12, Id = 23, CelebrityId = 12, Date = 24.12.2008 0:00:00, Description = Дата смерти, ReqPhotoPath =
----- GetCelebrityByLifeeventId -----
OK: Id = 12, FullName = Samuel Phillips Huntington, Nationality = US, ReqPhotoPath = Huntington.jpg

```

8. Проверьте с помощью теста работоспособность функций репозитория (***DAL_Celebrity_MSSQL***) и добейтесь успешного выполнения теста.

Задание 4

9. Фотографии, которые далее будут использоваться в приложениях, размещены **diskstation** (student/fitfit)



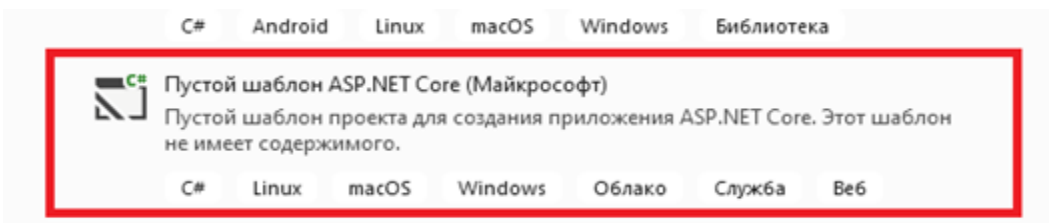
Название	Р...	Тип файла	Дата изменения
Babbage.jpg	3...	JPG Файл	2025-02-23 00:10:59
Berners-Lee.jpg	1...	JPG Файл	2024-11-08 18:38:56
Celebrities.json	6...	JSON Файл	2025-02-17 01:58:51
Chomsky.jpg	8...	JPG Файл	2024-11-08 18:39:02
Codd.jpg	4...	JPG Файл	2024-11-08 18:38:25
Dijkstra.jpg	2...	JPG Файл	2025-02-17 01:38:54
Einstein.jpg	4...	JPG Файл	2025-03-30 14:16:09
Ershov.jpg	6...	JPG Файл	2025-04-04 01:08:01
Huntington.jpg	6...	JPG Файл	2025-03-30 15:26:54
Knuth.jpg	4...	JPG Файл	2024-11-08 18:38:32
Linus.jpg	3...	JPG Файл	2024-11-08 18:38:41
Lovelace.jpg	1...	JPG Файл	2025-02-22 22:31:28
Neumann.jpg	3...	JPG Файл	2024-11-08 18:39:11
Tanenbaum.jpg	3...	JPG Файл	2025-02-23 00:05:43

10. Разработайте ASPA, применив следующий шаблон.

Имя решения: **ASPA**

Имя проекта: **ASPA006_1**

Версия .NET: **7.0** или **8.0**



11. Подключите библиотеку **DAL_Celebrity_MSSQL** к **ASPA006_1**.

12. Создайте конфигурационный файл **Celebrities.config.json**, позволяющий параметризовать директорию для фотографий и строку подключения к серверу СУБД.

```
{
  // Celebrities.config.json
  "Celebrities": {
    "PhotosFolder": "D:\\VS_Projects\\ANC\\ANC25_OpenAPI\\Photos",
    "ConnectionString": "Data source=██████████; Initial Catalog=██████████; TrustServerCertificate=True; User Id=██████████; Password=██████████"
  }
}
```

13. Обеспечьте доступ к конфигурации (строка подключения и папка для фотографий) с помощью объекта класса **CelebritiesConfig** интерфейса **IOptions**.

14. Создайте Soped-сервис, реализующий интерфейс **IRepository** (задание 2), как это сделано в следующем примере.

```
builder.Services.AddScoped<IRepository, Repository>((IServiceProvider p) => {  
    CelebritiesConfig config = p.GetRequiredService<IOptions<CelebritiesConfig>>().Value;  
    return new Repository(config.ConnectionString);  
});
```

15. Разработайте следующие конечные точки использующие сервис, реализующий **IRepository**:

```
// ----- ЗНАМЕНИТОСТИ (Celebrities) -----  
var celebrities = app.MapGroup("/api/Celebrities");  
// все знаменитости  
celebrities.MapGet("/", (IRepository repo) => repo.GetAllCelebrities());  
// знаменитость по ID  
celebrities.MapGet("/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// знаменитость по ID события  
celebrities.MapGet("/Lifeevents/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// удалить знаменитость по ID  
celebrities.MapDelete("/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// добавить новую знаменитость  
celebrities.MapPost("/", (IRepository repo, Celebrity celebrity) => ...);  
// изменить знаменитость по ID  
celebrities.MapPut("/{id:int:min(1)}", (IRepository repo, int id, Celebrity celebrity) => ...);  
// получить файл фотографии по имени файла (fname)  
celebrities.MapGet("/photo/{fname}", async (IOptions<CelebritiesConfig> iconfig, HttpContext context, string fname) => ...);
```

```
// ----- СОБЫТИЯ (Lifeevents) -----  
var lifeevents = app.MapGroup("/api/Lifeevents");  
// все события  
lifeevents.MapGet("/", (IRepository repo) => repo.GetAllLifeevents());  
// событие по ID  
lifeevents.MapGet("/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// все события по ID знаменитости  
lifeevents.MapGet("/Celebrities/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// удалить событие по ID  
lifeevents.MapDelete("/{id:int:min(1)}", (IRepository repo, int id) => ...);  
// добавить новое событие  
lifeevents.MapPost("/", (IRepository repo, Lifeevent lifeevent) => ...);  
// изменить событие по ID  
lifeevents.MapPut("/{id:int:min(1)}", (IRepository repo, int id, Lifeevent lifeevent) => ...);
```

16. Для обработки ошибок в конечных точках разработайте middleware-обработчик ошибок.

17. Тест (файл **app.http**, прилагается) для конечных точек.

```
###  
@hp=http://localhost:5204  
@capi=/api/Celebrities  
###
```

```
### все знаменитости  
### celebrities.MapGet("/",  
GET {{hp}}{{capi}}
```



```
Content:
[
  {
    "id": 1,
    "fullName": "Noam Chomsky",
    "nationality": "US",
    "reqPhotoPath": "Chomsky.jpg"
  },
  {
    "id": 2,
    "fullName": "Tim Berners-Lee",
    "nationality": "UK",
    "reqPhotoPath": "Berners-Lee.jpg"
  },
  {
    "id": 3,
    "fullName": "Edgar Codd",
    "nationality": "US",
    "reqPhotoPath": "Codd.jpg"
  },
  {
    "id": 4,
    "fullName": "Donald Knuth",
    "nationality": "US",
    "reqPhotoPath": "Knuth.jpg"
  },
  {
    "id": 5,
    "fullName": "Linus Torvalds",
    "nationality": "US",
    "reqPhotoPath": "Linus.jpg"
  },
  {
    "id": 6,
    "fullName": "John Neumann",
    "nationality": "US",
    "reqPhotoPath": "Neumann.jpg"
  },
]
```

```
### знаменитость по ID
### MapGet("/{id:int:min(1)}"
GET {{hp}}{{capi}}/7
```

```
Content:
{
  "id": 7,
  "fullName": "Edsger Dijkstra",
  "nationality": "NL",
  "reqPhotoPath": "Dijkstra.jpg"
}
```

```
###
### знаменитость по ID события
### MapGet("/Lifeevents/{id:int:min(1)}"
GET {{hp}}{{capi}}/Lifeevents/7
```

```
Content:
{
  "id": 4,
  "fullName": "Donald Knuth",
  "nationality": "US",
  "reqPhotoPath": "Knuth.jpg"
}
```

```
### удалить знаменитость по ID
###.MapDelete("/{id:int:min(1)}",
DELETE {{hp}}{{capi}}/4
```

Content:

```
{
  "id": 4,
  "fullName": "Donald Knuth",
  "nationality": "US",
  "reqPhotoPath": "Knuth.jpg"
}
```

```
###  удалить знаменитость по ID
###.MapDelete("/{id:int:min(1)}",
DELETE {{hp}}{{capi}}/4
```

Content:

```
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Not Found",
  "status": 404,
  "detail": "404002:Celebrity Id = 4",
  "instance": "ANC25"
}
```

```
###  добавить новую знаменитость
###  MapPost("/",
POST {{hp}}{{capi}}
Content-Type:application/json

{
  "fullName":      "Ершов Андрей",
  "nationality": "RU",
  "reqPhotoPath": "Ershov.jpg"
}
```

Content:

```
{
  "id": 13,
  "fullName": "Ершов Андрей",
  "nationality": "RU",
  "reqPhotoPath": "Ershov.jpg"
}
```

```
###  изменить знаменитость по ID
###  MapPut("/{id:int:min(1)}",
PUT {{hp}}{{capi}}/13
Content-Type:application/json

{
  "fullName":      "Ершов Андрей Петрович",
  "nationality": "RU",
  "reqPhotoPath": "Ershov.jpg"
}
```

```
{
  "id": 13,
  "fullName": "Ершов Андрей Петрович",
  "nationality": "RU",
  "reqPhotoPath": "Ershov.jpg"
}
```

```
### получить файл фотографии по имени файла (fname)
### MapGet("/photo/{fname}"
GET {{hp}}{{capi}}/photo/Ershov.jpg
```

????JFIF,,??C

```
'@"$'92<;8276?GZL?CUD67NkOU' efe=KownbvZceaC..aA7Aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa','  
? [<00[0;0? p0000I0 000'  
00n,00H0j00iF000%A0000I0:"`  
0]0w060-00w00C3;000000{0i"EH0h(00<|C00800T0&T0  
C00087EX000r006D0w0yw 0`s  
0 p{Au'0x0000;0}?0Qb00#z,<Q00000d0n0M0;Z0RKq00Ah0&00  
ËN00L/05008000+0`i'L-00000RF00&0+uș0g800ZvH0MĖ.0Vo00#00000Y00z00bEjP600&L0S00V00000&00<0+Z
```

###

```
### все события
### MapGet("/",
GET {{hp}}{{lapi}}
```

Content:

```
[
  {
    "id": 1,
    "celebrityId": 1,
    "date": "1928-12-07T00:00:00",
    "description": "Дата рождения",
    "reqPhotoPath": null
  },
  {
    "id": 2,
    "celebrityId": 1,
    "date": "1955-01-01T00:00:00",
    "description": "Издание книги \"Логическая структура лингвистической теории\"",
    "reqPhotoPath": null
  },
  {
    "id": 3,
    "celebrityId": 2,
    "date": "1955-06-08T00:00:00",
    "description": "Дата рождения",
    "reqPhotoPath": null
  },
  {
    "id": 4,
    "celebrityId": 2,
    "date": "1989-06-08T00:00:00",
    "description": "В CERN предложил \"Гиппертекстовый проект\"",
    "reqPhotoPath": null
  },
  {
    "id": 5,
    "celebrityId": 3,
    "date": "1923-08-23T00:00:00",
    "description": "Дата рождения",
    "reqPhotoPath": null
  },
  {
    "id": 6,
    "celebrityId": 3,
    "date": "2003-04-18T00:00:00",
    "description": "Дата рождения"
  }
]
```

```
### все события
### MapGet("/",
GET {{hp}}{{lapi}}
```

Content:

```
{
  "id": 10,
  "celebrityId": 5,
  "date": "1991-09-17T00:00:00",
  "description": "Выложил исходный код OS Linus (версии 0.01)",
  "reqPhotoPath": null
}
```

```
### все события по ID знаменитости
### MapGet("/Celebrities/{id:int:min(1)}
GET {{hp}}{{lapi}}/Celebrities/3
```

```
Content:
[
  {
    "id": 5,
    "celebrityId": 3,
    "date": "1923-08-23T00:00:00",
    "description": "Дата рождения",
    "reqPhotoPath": null
  },
  {
    "id": 6,
    "celebrityId": 3,
    "date": "2003-04-18T00:00:00",
    "description": "Дата смерти",
    "reqPhotoPath": null
  }
]
```

```
### удалить событие по ID
### MapDelete("/{id:int:min(1)}"
DELETE {{hp}}{{lapi}}/10
```

```
Content:
{
  "id": 10,
  "celebrityId": 5,
  "date": "1991-09-17T00:00:00",
  "description": "Выложил исходный код OS Linus (версии 0.01)",
  "reqPhotoPath": null
}
```

```
### удалить событие по ID
### MapDelete("/{id:int:min(1)}"
DELETE {{hp}}{{lapi}}/10
```

```
Content:
{
  "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4",
  "title": "Not Found",
  "status": 404,
  "detail": "404004:Lifevent Id = 10",
  "instance": "ANC25"
}
```

```
### добавить новое событие
### MapPost("/",
POST {{hp}}{{lapi}}
Content-Type:application/json
{
  "celebrityId": 13,
  "date": "1931-04-19T00:00:00",
  "description": "Дата рождения",
  "reqPhotoPath": null
}
```



```
Content:
{
  "id": 24,
  "celebrityId": 13,
  "date": "1931-04-19T00:00:00",
  "description": "Дата рождения",
  "reqPhotoPath": null
}
```

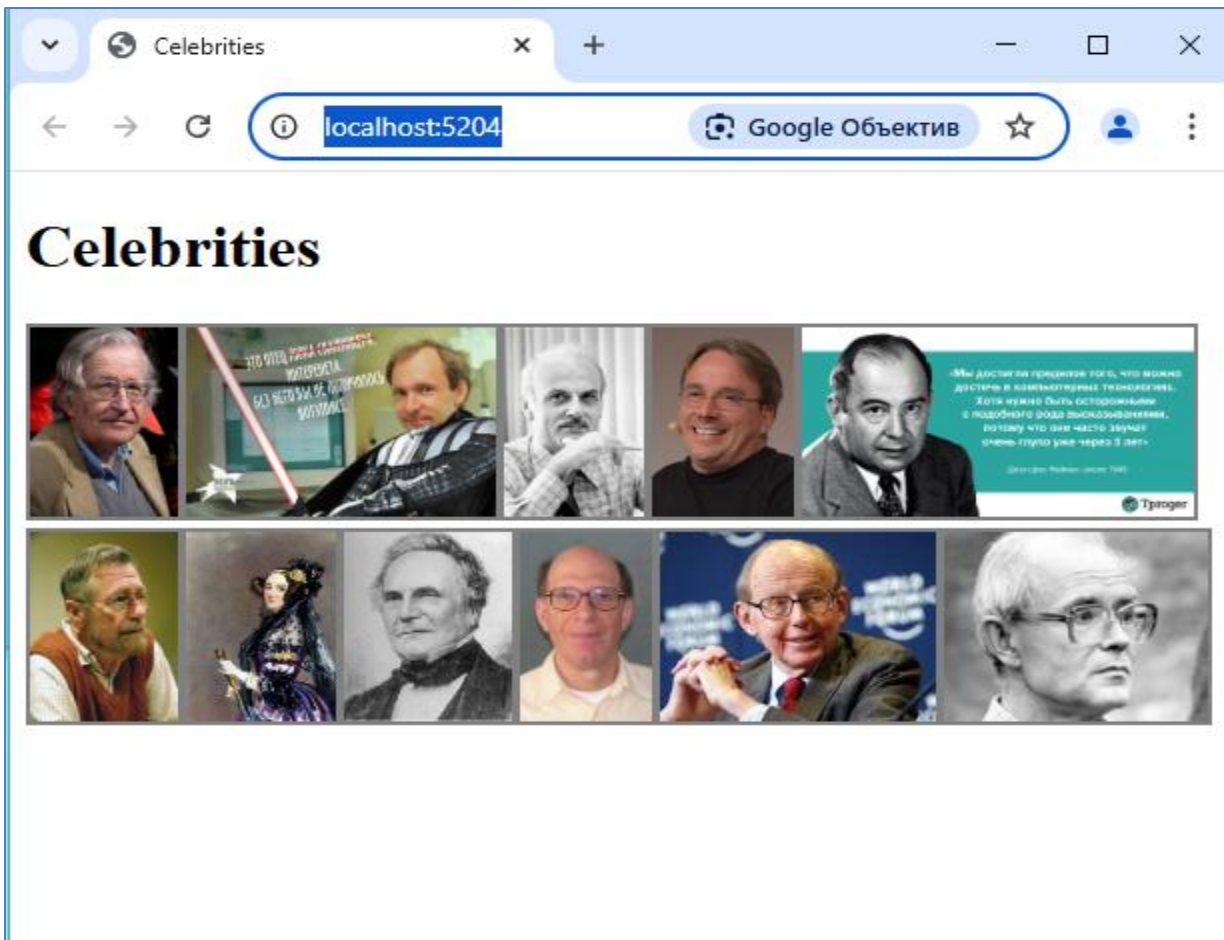
```
###  добавить новое событие
###  MapPost("/",
POST {{hp}}{{lapi}}
Content-Type:application/json

{
  "celebrityId": 13,
  "date": "1931-04-19T00:00:00",
  "description": "Дата рождения",
  "reqPhotoPath": null
}
```

```
Content:
{
  "id": 24,
  "celebrityId": 13,
  "date": "1931-04-19T00:00:00",
  "description": "Дата рождения",
  "reqPhotoPath": null
}
```

18. Разработайте html-страницу ***Index.html*** следующего вида. Обеспечьте отображение этой страницы при запуске приложения по умолчанию. Ниже приводится внешний вид страницы при запуске приложения.

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8" />
  <title>Celebrities</title>
  <script src="/js/Celebrities.js" defer></script>
</head>
<body>
  <h1>Celebrities</h1>
</body>
</html>
```



19. При одинарном клике мышью фотографии, внизу отображается, строки с данными о событиях.

